

Práctica M27 - Agrupación mediante K-Means

Análisis de Clustering con Reducción de Dimensionalidad PCA

Alumno: RobertScience

Programa: Profesión Científico de Datos v2

Herramientas utilizadas: Python, Pandas, Scikit-Learn, Matplotlib

Tipo de análisis: Machine Learning No Supervisado

Descripción del proyecto

El objetivo de esta práctica es aplicar técnicas de **aprendizaje no supervisado** para identificar patrones de agrupamiento dentro de un conjunto de datos.

Para este ejercicio se utilizará el dataset **Iris**, uno de los conjuntos de datos más conocidos en el área de ciencia de datos y aprendizaje automático. Este dataset contiene mediciones de distintas características de flores del género *Iris*.

El análisis se realizará utilizando el algoritmo **K-Means**, el cual permite agrupar observaciones en clusters basados en la similitud entre sus características.

Durante el desarrollo del análisis se realizarán los siguientes pasos:

- Descarga y carga del dataset desde una fuente en línea
- Exploración inicial de los datos
- Determinación del número óptimo de clusters
- Aplicación del algoritmo K-Means
- Reducción de dimensionalidad mediante **PCA**
- Comparación entre el clustering con datos originales y con datos transformados

Introducción

El **clustering** es una técnica de aprendizaje automático no supervisado que permite identificar grupos naturales dentro de un conjunto de datos.

A diferencia de los modelos supervisados, en los cuales existen etiquetas previamente definidas, los algoritmos de clustering buscan descubrir estructuras ocultas en los datos sin información previa sobre las clases.

Uno de los algoritmos más utilizados para este propósito es **K-Means**, el cual divide un conjunto de observaciones en k grupos de manera que cada observación pertenece al cluster cuyo centroide es más cercano.

En esta práctica se utilizará el dataset Iris para aplicar el algoritmo K-Means y analizar cómo la reducción de dimensionalidad mediante **Análisis de Componentes Principales (PCA)** puede afectar los resultados del agrupamiento.

Objetivo del análisis

El objetivo principal de esta práctica es aplicar técnicas de clustering para analizar la estructura del dataset Iris.

Los objetivos específicos del análisis son:

1. Descargar y cargar el dataset Iris desde una fuente en línea.
2. Aplicar el algoritmo **K-Means** utilizando las variables originales.
3. Determinar el número óptimo de clusters mediante:
 - Método del **Codo (Elbow Method)**
 - **Coficiente Silhouette**
4. Aplicar una reducción de dimensionalidad mediante **PCA**.
5. Repetir el proceso de clustering utilizando las dimensiones reducidas.
6. Comparar los resultados obtenidos y analizar las ventajas del uso de PCA antes de aplicar K-Means.

Descarga y carga del dataset

Para este análisis se utilizará el dataset **Iris**, el cual contiene mediciones de tres especies diferentes de flores:

- Setosa
- Versicolor
- Virginica

Las variables incluidas en el dataset son:

- **sepal.length** → longitud del sépalo
- **sepal.width** → ancho del sépalo
- **petal.length** → longitud del pétalo
- **petal.width** → ancho del pétalo
- **variety** → especie de la flor

El dataset será cargado directamente desde una fuente en línea utilizando la librería **Pandas**.

```
In [1]: # Importar Librerías necesarias

import pandas as pd

# URL del dataset Iris
url = "https://gist.githubusercontent.com/netj/8836201/raw/iris.csv"

# Cargar dataset
df = pd.read_csv(url)

# Mostrar las primeras filas
df.head()
```

Out[1]:

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa

Exploración inicial de los datos

Una vez cargado el dataset, es importante realizar una inspección inicial para entender su estructura.

Para ello se muestran las primeras filas del DataFrame, lo cual permite verificar:

- las variables disponibles
- el tipo de datos
- el formato general del dataset

Este paso es fundamental dentro del proceso de análisis exploratorio de datos (**EDA**).

Dimensiones del dataset

Antes de comenzar el análisis es importante conocer el tamaño del conjunto de datos.

Para ello se revisa:

- Número de observaciones (filas)
- Número de variables (columnas)

Esto permite entender la escala del dataset con el que se trabajará durante el proceso de clustering.

```
In [2]: # Dimensiones del dataset

df.shape
```

Out[2]: (150, 5)

Tipos de datos

El siguiente paso consiste en analizar los tipos de datos presentes en el dataset.

Esto es importante ya que los algoritmos de clustering como **K-Means** requieren trabajar con variables numéricas.

```
In [3]: # Información general del dataset

df.info()

<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   sepal.length    150 non-null   float64
1   sepal.width     150 non-null   float64
2   petal.length    150 non-null   float64
3   petal.width     150 non-null   float64
4   variety         150 non-null   str
dtypes: float64(4), str(1)
memory usage: 6.0 KB
```

Estadísticas descriptivas

Se calculan estadísticas descriptivas de las variables numéricas para comprender mejor la distribución de los datos.

Entre las métricas analizadas se encuentran:

- Media
- Desviación estándar
- Valores mínimos
- Cuartiles
- Valores máximos

Este análisis permite entender el rango y la variabilidad de las mediciones presentes en el dataset.

In [4]: *# Estadísticas descriptivas*

```
df.describe()
```

Out[4]:

	sepal.length	sepal.width	petal.length	petal.width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

Visualización de las variables

Para comprender mejor la distribución de los datos, se generan gráficos de dispersión entre algunas de las variables más representativas del dataset.

Esto permite identificar posibles patrones naturales de agrupamiento entre las observaciones.

In [5]: *import* matplotlib.pyplot *as* plt

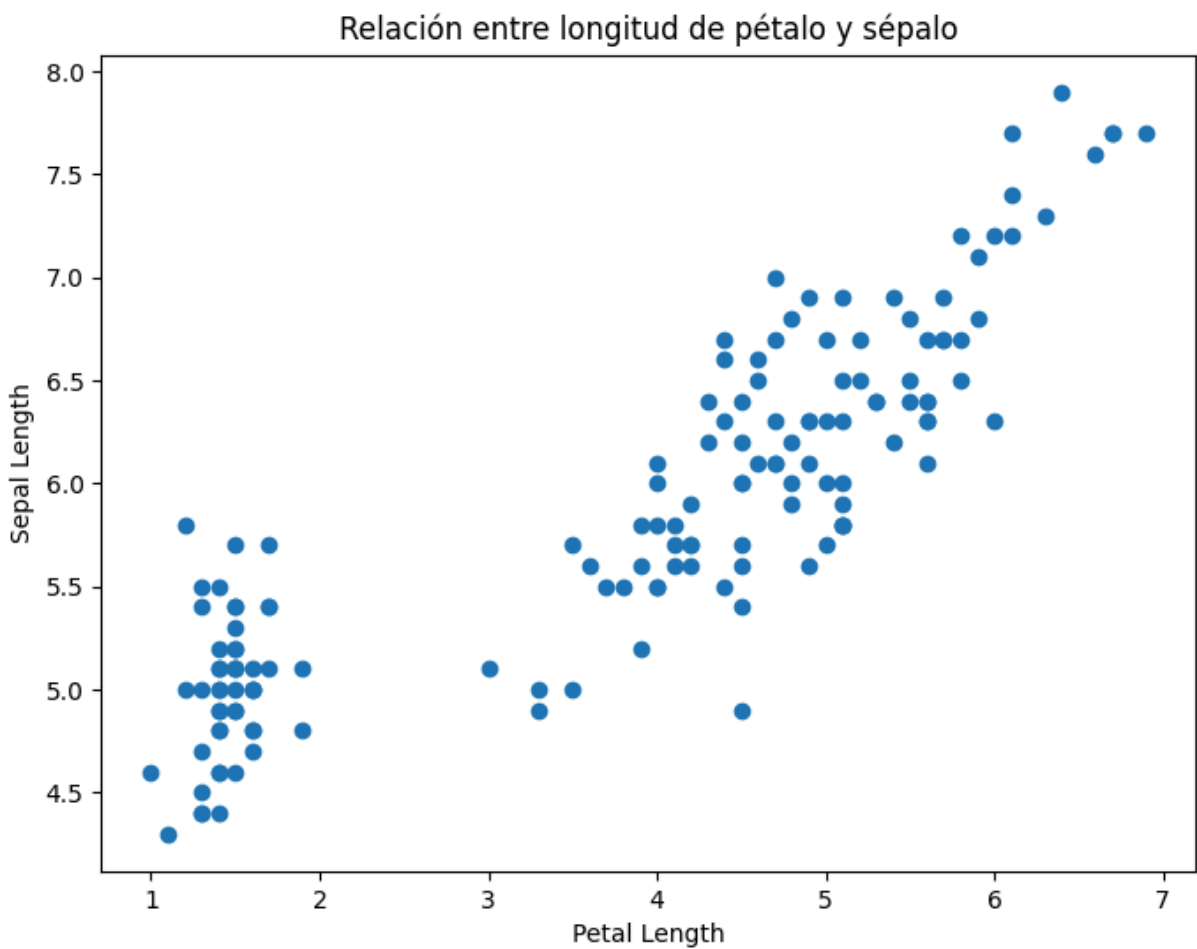
Gráfico de dispersión entre Longitud de pétalo y Longitud de sépalo

```
plt.figure(figsize=(8,6))

plt.scatter(
    df["petal.length"],
    df["sepal.length"]
)

plt.xlabel("Petal Length")
plt.ylabel("Sepal Length")
plt.title("Relación entre longitud de pétalo y sépalo")

plt.show()
```



Preparación de los datos para clustering

El algoritmo **K-Means** requiere trabajar únicamente con variables numéricas.

Por esta razón se eliminan las variables categóricas del dataset, manteniendo únicamente las variables que representan mediciones numéricas de las flores.

Las variables utilizadas para el clustering serán:

- sepal.length
- sepal.width
- petal.length
- petal.width

```
In [6]: # Seleccionar variables numéricas

X = df.drop("variety", axis=1)

# Mostrar primeras filas
X.head()
```

Out[6]:

	sepal.length	sepal.width	petal.length	petal.width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Determinación del número óptimo de clusters

Antes de aplicar el algoritmo K-Means es necesario determinar el número adecuado de clusters (k).

Seleccionar un valor incorrecto de k puede generar agrupamientos poco representativos de la estructura real de los datos.

Para identificar el número óptimo de clusters se utilizarán dos métodos:

1. **Método del Codo (Elbow Method)**

Analiza cómo cambia la suma de distancias dentro de los clusters a medida que aumenta el número de grupos.

2. **Coeficiente Silhouette**

Mide qué tan bien se separan los clusters entre sí, evaluando la cohesión interna y la separación entre grupos.

Método del Codo (Elbow Method)

El método del codo consiste en ejecutar el algoritmo K-Means con diferentes valores de k y calcular la **inercia** (suma de distancias cuadradas dentro de los clusters).

A medida que aumenta el número de clusters, la inercia disminuye. Sin embargo, llega un punto en el cual la reducción comienza a ser menos significativa.

Ese punto de inflexión es conocido como **“el codo”** y sugiere el número óptimo de clusters.

```
In [7]: from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

inertia = []
K_range = range(1,11)

for k in K_range:

    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X)

    inertia.append(kmeans.inertia_)

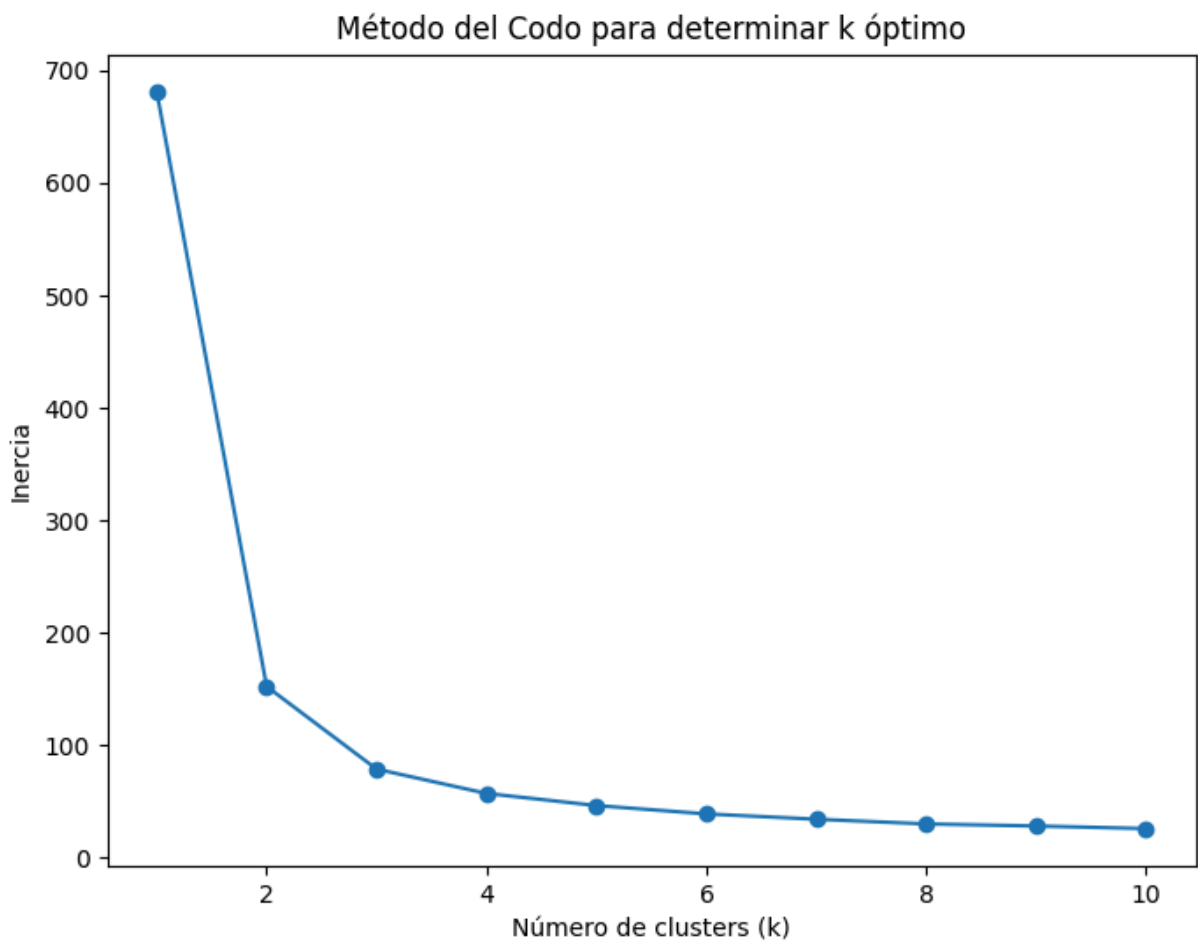
# Gráfico del método del codo

plt.figure(figsize=(8,6))

plt.plot(K_range, inertia, marker='o')

plt.xlabel("Número de clusters (k)")
plt.ylabel("Inercia")
plt.title("Método del Codo para determinar k óptimo")

plt.show()
```



Evaluación mediante el coeficiente Silhouette

El coeficiente Silhouette mide qué tan bien se agrupan las observaciones dentro de su cluster en comparación con otros clusters.

Este valor varía entre:

- **-1** → agrupamiento incorrecto
- **0** → clusters superpuestos
- **1** → clusters bien definidos

El valor de k que produce el **mayor coeficiente Silhouette** suele considerarse el número óptimo de clusters.

```
In [8]: from sklearn.metrics import silhouette_score

silhouette_scores = []

K_range = range(2,11)

for k in K_range:

    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X)

    score = silhouette_score(X, labels)

    silhouette_scores.append(score)

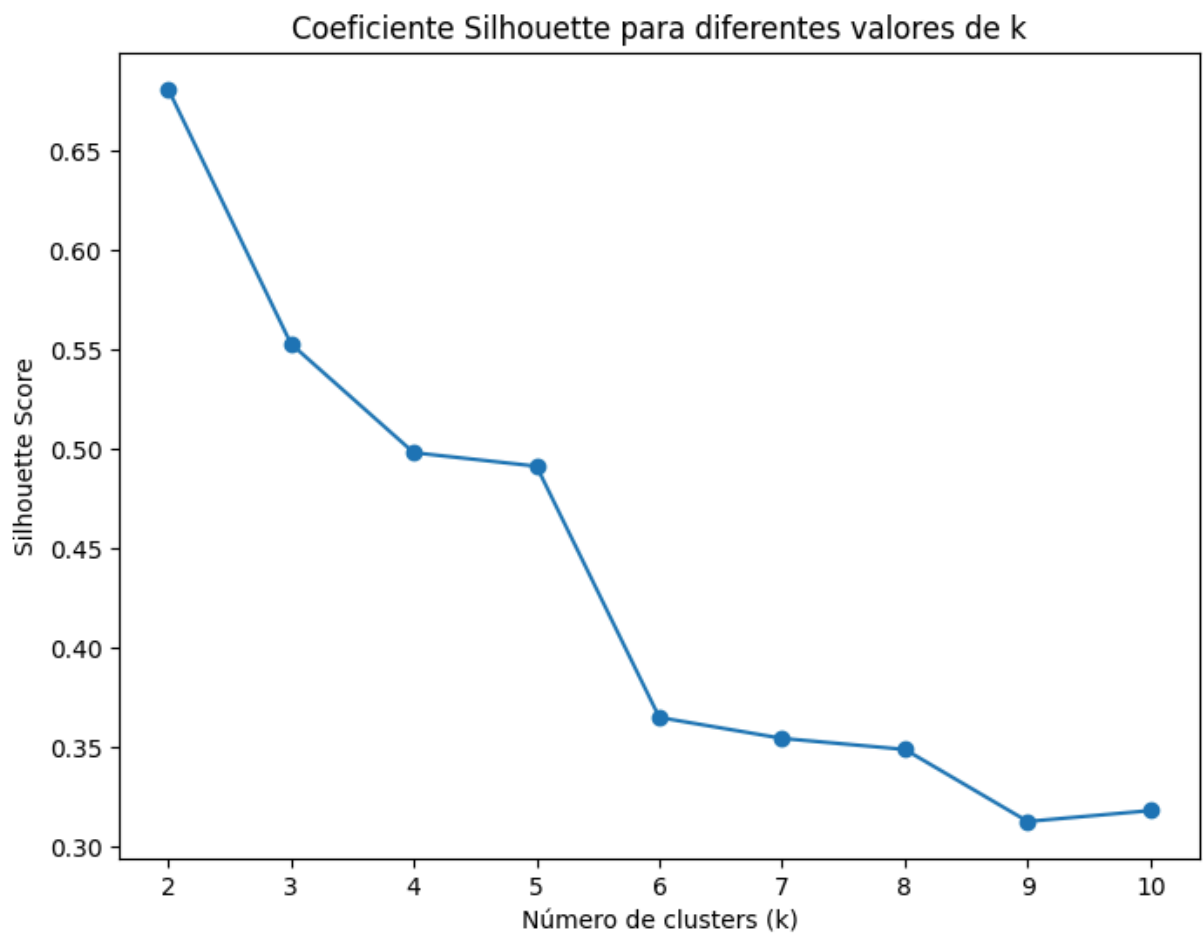
# Gráfico Silhouette

plt.figure(figsize=(8,6))

plt.plot(K_range, silhouette_scores, marker='o')

plt.xlabel("Número de clusters (k)")
plt.ylabel("Silhouette Score")
plt.title("Coeficiente Silhouette para diferentes valores de k")

plt.show()
```



Interpretación de resultados

A partir de los resultados obtenidos mediante el método del codo y el coeficiente Silhouette, se puede identificar un número adecuado de clusters para el dataset.

El gráfico del método del codo muestra un punto de inflexión alrededor de **k = 3**, lo que indica que agregar más clusters después de este punto no mejora significativamente la reducción de la inercia.

De manera complementaria, el análisis mediante el coeficiente Silhouette muestra valores altos para **k = 3**, lo cual indica una buena separación entre los grupos.

Por lo tanto, se selecciona **k = 3** como el número óptimo de clusters para aplicar el algoritmo K-Means en este dataset.

Aplicación del algoritmo K-Means

Una vez determinado el número óptimo de clusters mediante los métodos del codo y Silhouette, se procede a aplicar el algoritmo **K-Means** al conjunto de datos original.

El algoritmo K-Means busca agrupar las observaciones en un número determinado de clusters, minimizando la distancia entre los puntos y el centroide de cada grupo.

Para este análisis se utilizará **k = 3**, ya que los métodos de evaluación sugieren que este valor representa adecuadamente la estructura del dataset.

```
In [9]: # Aplicar K-Means con k = 3

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)

clusters = kmeans.fit_predict(X)

# Agregar clusters al dataset
df["cluster"] = clusters

df.head()
```


Out[9]:

	sepal.length	sepal.width	petal.length	petal.width	variety	cluster
0	5.1	3.5	1.4	0.2	Setosa	1
1	4.9	3.0	1.4	0.2	Setosa	1
2	4.7	3.2	1.3	0.2	Setosa	1
3	4.6	3.1	1.5	0.2	Setosa	1
4	5.0	3.6	1.4	0.2	Setosa	1

Visualización de los clusters obtenidos

Para comprender mejor el resultado del algoritmo K-Means, se genera una visualización de dispersión utilizando dos de las variables más representativas del dataset.

Esta gráfica permite observar cómo las observaciones se agrupan en distintos clusters de acuerdo con sus características.

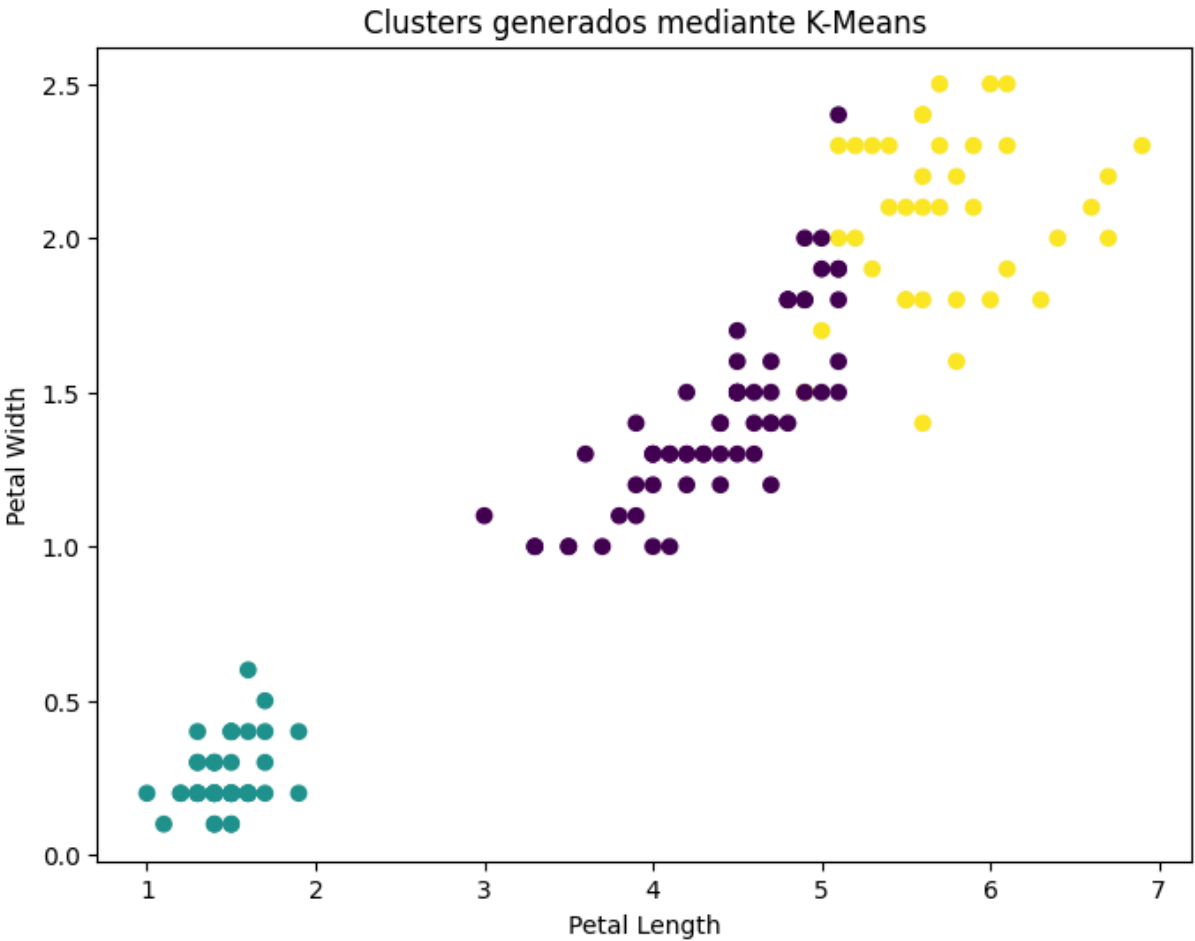
In [10]:

```
plt.figure(figsize=(8,6))

plt.scatter(
    df["petal.length"],
    df["petal.width"],
    c=df["cluster"],
    cmap="viridis"
)

plt.xlabel("Petal Length")
plt.ylabel("Petal Width")
plt.title("Clusters generados mediante K-Means")

plt.show()
```



Interpretación del agrupamiento

Los resultados obtenidos mediante el algoritmo K-Means muestran la presencia de tres grupos claramente diferenciados dentro del dataset.

Estos clusters representan agrupaciones naturales basadas en las características morfológicas de las flores.

Aunque el algoritmo no utiliza la variable de especie (*variety*), los clusters obtenidos suelen coincidir en gran medida con las tres especies presentes en el dataset:

- Iris Setosa
- Iris Versicolor
- Iris Virginica

Esto demuestra que las variables numéricas del dataset contienen suficiente información para identificar patrones de agrupamiento naturales.

Reducción de dimensionalidad mediante PCA

En muchos problemas de ciencia de datos, los datasets pueden contener una gran cantidad de variables. Esto puede dificultar la visualización y el análisis de los datos.

Para resolver este problema se utiliza **PCA (Principal Component Analysis)**, una técnica de reducción de dimensionalidad que permite transformar un conjunto de variables originales en un número menor de componentes que conservan la mayor parte de la información del dataset.

En este ejercicio se reducirá el dataset original de **cuatro variables a dos componentes principales**, lo que permitirá visualizar los datos en un plano bidimensional.

```
In [11]: from sklearn.decomposition import PCA

# Crear modelo PCA con 2 componentes
pca = PCA(n_components=2)

# Transformar los datos
X_pca = pca.fit_transform(X)

# Convertir a DataFrame
pca_df = pd.DataFrame(X_pca, columns=["PC1", "PC2"])

pca_df.head()
```

Out[11]:

	PC1	PC2
0	-2.684126	0.319397
1	-2.714142	-0.177001
2	-2.888991	-0.144949
3	-2.745343	-0.318299
4	-2.728717	0.326755

```
In [12]: print("Varianza explicada por cada componente:")
print(pca.explained_variance_ratio_)
```

Varianza explicada por cada componente:
[0.92461872 0.05306648]

Componentes principales obtenidos

Los componentes principales representan combinaciones lineales de las variables originales del dataset.

Estas nuevas variables permiten concentrar la mayor cantidad de información posible en un menor número de dimensiones.

En este caso se utilizarán dos componentes principales:

- **PC1 (Primer componente principal)**
- **PC2 (Segundo componente principal)**

Estas dos dimensiones permitirán visualizar la estructura del dataset y aplicar el algoritmo de clustering en un espacio reducido.

```
In [13]: # Aplicar K-Means a los datos reducidos

kmeans_pca = KMeans(n_clusters=3, random_state=42, n_init=10)

clusters_pca = kmeans_pca.fit_predict(pca_df)

# Agregar clusters al dataframe PCA
pca_df["cluster"] = clusters_pca

pca_df.head()
```

Out[13]:

	PC1	PC2	cluster
0	-2.684126	0.319397	1
1	-2.714142	-0.177001	1
2	-2.888991	-0.144949	1
3	-2.745343	-0.318299	1
4	-2.728717	0.326755	1

Visualización de clusters en el espacio PCA

Una vez aplicados PCA y K-Means, se genera una visualización en dos dimensiones utilizando los componentes principales.

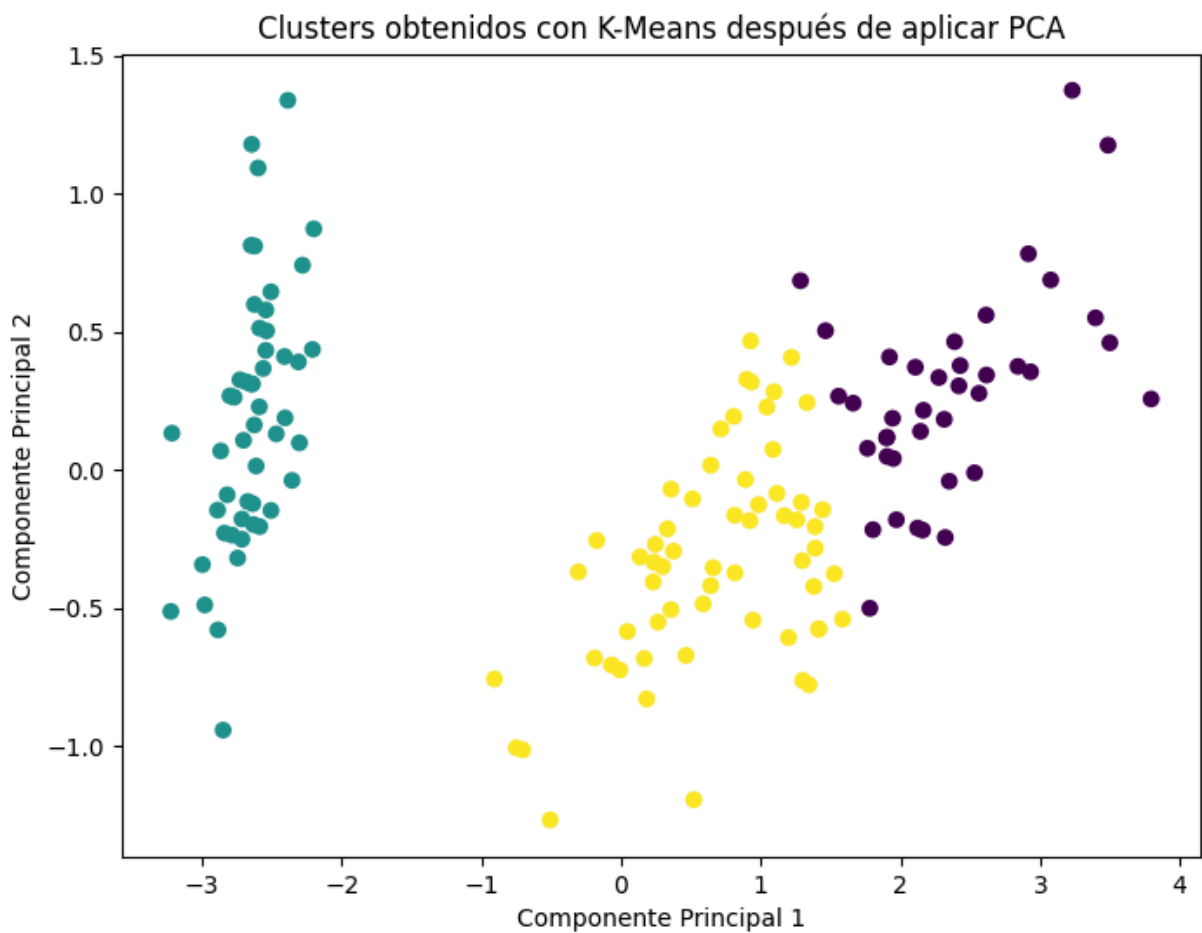
Esta gráfica permite observar de forma clara la separación entre los distintos clusters generados por el algoritmo.

```
In [14]: plt.figure(figsize=(8,6))

plt.scatter(
    pca_df["PC1"],
    pca_df["PC2"],
    c=pca_df["cluster"],
    cmap="viridis"
)

plt.xlabel("Componente Principal 1")
plt.ylabel("Componente Principal 2")
plt.title("Clusters obtenidos con K-Means después de aplicar PCA")

plt.show()
```



Comparación de resultados

Al comparar los resultados obtenidos mediante K-Means con los datos originales y con los datos transformados mediante PCA, se observa que los clusters identificados son muy similares en ambos casos.

Esto se debe a que el dataset Iris tiene una estructura relativamente simple y bien definida, por lo que el algoritmo puede identificar correctamente los grupos incluso sin reducción de dimensionalidad.

Sin embargo, la aplicación de PCA ofrece una ventaja importante: permite **visualizar los clusters de manera clara en un espacio bidimensional**, lo cual facilita la interpretación de los resultados.

Además, en datasets más grandes o con muchas variables, la reducción de dimensionalidad puede mejorar la eficiencia computacional y ayudar a eliminar ruido o redundancia en los datos.

Conclusiones

En esta práctica se aplicó el algoritmo de clustering K-Means al dataset Iris para identificar agrupamientos naturales dentro de los datos.

Mediante el método del codo y el coeficiente Silhouette se determinó que **tres clusters** representan adecuadamente la estructura del dataset.

Posteriormente se aplicó una reducción de dimensionalidad mediante **PCA**, lo que permitió representar los datos en dos dimensiones sin perder una parte significativa de la información original.

Los resultados obtenidos muestran que el algoritmo K-Means es capaz de identificar correctamente los patrones presentes en el dataset, y que el uso de PCA facilita la visualización y la interpretación de los clusters generados.

Este tipo de técnicas son ampliamente utilizadas en ciencia de datos para la **segmentación de clientes, análisis de patrones y descubrimiento de estructuras ocultas en los datos.**